

(CONTEXT FIRST)

This project implements a lightweight EDR-style monitoring agent that collects endpoint and network telemetry, applies rule-based detection, correlates events across multiple sources, and forwards alerts to a SIEM system for analysis and visualization.

Collectors → Detection → Correlation → Output (SIEM)

The system works like this:

Collect data → Analyze → Detect threats → Correlate activity → Send alerts

collectors/windows_logs.py

Purpose

Collect security events from Windows Event Logs

What the code does

- Connects to Windows Event Log system
 - Reads recent **Security logs**
 - Extracts:
 - Event ID (what happened)
 - Timestamp (when it happened)
-

Key Concepts

Event ID = type of activity (login, process, etc.)

Example:

- 4625 → Failed login
 - 4688 → Process created
-

Why this matters

This is the primary source of security events

Without logs:

- No detection
- No investigation

Simple Explanation

"Go to Windows Event Viewer → pull latest security events → return them as data"

collectors/process_monitor.py

Purpose

Track running processes on the system

What the code does

- Uses psutil
 - Loops through all active processes
 - Extracts:
 - PID (process ID)
 - PPID (parent process)
 - Name (program)
-

Key Concepts

PID = unique process ID

PPID = who launched the process

Why this matters

Detect suspicious chains like:

winword.exe → powershell.exe → malware

Simple Explanation

"List all programs currently running and who started them"

collectors/network_monitor.py

Purpose

Capture active network connections

What the code does

- Uses psutil
 - Retrieves active network sessions
 - Extracts:
 - Source IP + port
 - Destination IP + port
 - Connection status
-

Key Concepts

src_ip → your machine

dst_ip → external system

Why this matters

Detect:

- ✓ Data exfiltration
 - ✓ Reverse shells
 - ✓ Suspicious outbound traffic
-

Simple Explanation

"Show which programs are talking to which external systems"

detection/rules.py

Purpose

Apply detection logic to identify suspicious activity

What the code does

- Loops through all collected events
 - Checks for known patterns
 - Generates alerts when conditions match
-

Detection Examples

Brute force

IF Event ID = 4625 → Failed login
THEN → Alert: BRUTE_FORCE

Suspicious process

IF process = powershell.exe
THEN → Alert: suspicious execution

Reverse shell

IF port = 4444
THEN → Alert: possible backdoor

Why this matters

This is your “brain” (decision-making layer)

Simple Explanation

"Look at data and decide if something is dangerous"

detection/correlator.py

Purpose

Link multiple events together to detect full attack chains

What the code does

- Compares:
 - Process activity
 - Network activity
 - Looks for relationships between them
-

Example logic

IF:

- powershell.exe is running

AND

- it is making external connection

THEN:

→ Possible attack chain

Why this matters

Single events are weak signals

Combined events = strong evidence

Example

Login failed → Process launched → Network connection

That's a full attack pattern

Simple Explanation

"Connect the dots between different activities"

sender/sender.py

Purpose

Send alerts to your SIEM system

What the code does

- Uses HTTP POST request
- Sends alerts to:

`http://localhost:3001/logs`

Key Concept

SIEM = central system that receives alerts

Why this matters

Separates:

Detection engine ≠ visualization system

Simple Explanation

"Send detected threats to a dashboard or server"

main.py

Purpose

Orchestrates the entire pipeline

What the code does (step-by-step)

1. Collect data

Logs + Processes + Network

2. Combine all data

One dataset for analysis

3. Run detection rules

Find suspicious activity

4. Run correlation

Find attack chains

5. Send alerts

Forward to SIEM

Why this matters

This is your pipeline controller

Simple Explanation"Run everything in order to detect threats"

requirements.txt

Purpose

List dependencies needed to run the project

Packages

- `pywin32` → Windows logs
 - `psutil` → process + network monitoring
 - `requests` → send alerts
-

Simple Explanation

"Install these libraries before running the project"

HOW EVERYTHING WORKS TOGETHER

Full Flow

1. Collect logs (Windows)
 2. Collect processes (PID/PPID)
 3. Collect network connections
 4. Apply detection rules
 5. Correlate activity
 6. Generate alerts
 7. Send to SIEM
-

FINAL UNDERSTANDING (FOR DOCUMENTATION)

System Summary

This project implements a lightweight EDR-style monitoring agent that collects endpoint and network telemetry, applies rule-based detection, correlates events across multiple sources, and forwards alerts to a SIEM system for analysis and visualization.